

Структурный вывод и типизация PydanticAI



Влад Прошинский
AI-консультант, ex-CPO

Обо мне

Влад Прошинский

AI-консультант, ex-CPO

- 10 лет в IT. Запускал IT продукты в Ozon, SOKOLOV, Деловые Линии, RUTUBE, Газпромбанке
- Сейчас помогаю компаниям внедрять AI так по отработанной методологии (Yakov & Partners Playbook) так, чтобы это давало измеримый результат.

Специализация:

- ИИ в маркетинге
- ИИ для отделов продаж
- Автоматизация бизнес-процессов



Канал «ИИ для продакта & CPO»
<https://t.me/AImademyday>

Программа

- Schema-first: типы и схемы вместо «парсинга JSON руками».
- Schema-guided reasoning.
- Dependencies: контекст/права/ключи через зависимости, а не через промпт.
- Validation loops: автоисправление ответа при провале валидации.

Результат: агент не ломает бэкенд неверным форматом данных

Что разберем:

01

Блок 1 — Проблема

Почему парсинг JSON «руками» — технический долг

02

Блок 2 — Schema-first

Типы как контракт между агентом и системой

03

Блок 3 — Dependencies

Контекст без промпта — права, ключи, данные

04

Блок 4 — Validation Loops

Авто-исправление при провале валидации

«Верни JSON» работает в 80–95% случаев

Оставшиеся 5–20% ждут тебя в production

80–95%

Prompt Engineering
«return JSON»

95–99%

Function Calling
/ Tool Use

100%

Native Structured
Output (constrained)*

 **Разница между 95% и 100%: 1 из 20 запросов тихо падает без ошибки**

- Level 1 (Prompt): тихие провалы на edge cases — нет гарантий типов
- Level 2 (Function calling): схема — это hint, не constraint
- Level 3 (Constrained decoding): FSM маскирует невалидные токены прямо при генерации

*На каждом шаге генерации LLM вычисляет вероятности для всех возможных токенов. При обычной генерации модель выбирает наиболее вероятный токен свободно. При constrained decoding определённые токены, нарушающие нужную схему (например, JSON Schema), просто запрещаются — модель физически не может их выдать.

Если чуть глубже

FSM — это Finite State Machine, в контексте генерации токенов LLM это работает специфически

Как FSM применяется при генерации токенов

Когда LLM генерирует текст — она выбирает следующий токен из словаря (десятки тысяч вариантов). Без ограничений модель может выбрать что угодно. При constrained decoding FSM накладывает маску на словарь перед каждым шагом генерации:

```
Схема ожидает: { "priority": "___" }  
Допустимые значения: "low", "medium",  
"high", "critical"
```

FSM в момент генерации:

- токен "low"	→ разрешён	✓
- токен "medium"	→ разрешён	✓
- токен "срочно"	→ заблокирован	✗
- токен "URGENT"	→ заблокирован	✗
- токен "8"	→ заблокирован	✗

Модель физически **не может выбрать запрещённый токен** — его вероятность обнуляется до того, как произошла выборка.

FSM / constrained decoding — невалидные токены не существуют в момент генерации

Разница как между «пожалуйста, не выходи за линию» и физическим забором.

В чем разница?

Метод	Как работает	Надёжность
Prompt Engineering	Просит модель ответить правильно	Низкая — модель может не слушаться
JSON Mode	Инструктирует модель писать валидный JSON	Средняя — структура может нарушаться
Native Structured Output (constrained)	Принудительно ограничивает генерацию токенов	Высокая — нарушение схемы физически невозможно

Инцидент: поле count вернулось как "three"

Как один edge case положил пайплайн в production

4 часа до обнаружения · 0 автотестов поймали · весь батч записан как null

- Модель вернула "count": "three" вместо "count": 3
- Бэкенд ожидал int → тихий TypeError → весь батч записан как null
- Обнаружили через 4 часа при ручной проверке отчёта
- Ни один unit-тест не поймал: модель в тестах всегда возвращала число
- Первопричина: промпт был инструкцией, не контрактом

Решение: схема — это контракт. Pydantic проверяет тип до того как данные попадут в код.

БЛОК 1 — ПАРСИНГ JSON

Три уровня контроля вывода LLM

Level 1 Prompt Engineering

«Return JSON with fields: name, email, score»

Работает в 80–95%

Тихие провалы на edge cases
Нет гарантий типов
Один промпт = один шанс

Level 2 Function Calling

Схема как hint при вызове функции

Работает в 95–99%

Схема — рекомендация,
а не constraint
Модель может нарушить

Level 3 Native Structured

Constrained decoding
с JSON Schema

Работает в 100%

State Machine маскирует
невалидные токены на уровне
генерации → PydanticAI
использует это

Пять способов, которыми LLM ломает ваш JSON

1

"count": "three"

строка вместо числа → тихий TypeError в downstream

2

Markdown-обёртка

````json\n{\n}\n```` → `json.loads()` падает на символах

3

**Отсутствует поле**

обязательное поле пропущено → NPE в следующем сервисе

4

**Лишние поля**

`"extra_field": ".."` → strict-режим ORM выбрасывает исключение

5

**Вложенный объект как строка**

`"address": "{ \"city\": \"Moscow\"}"` → невозможно индексировать

# Типичный «защитный» код без Pydantic — это бомба замедленного действия

Каждая строка — отдельная точка отказа:

- **data = json.loads(response)** — упадёт на markdown-обёртке
- **name = data.get("name", "")** — молчаливо пропустит отсутствие поля
- **age = int(data["age"])** — упадёт если "age": "unknown"
- **score = data["score"] \* 100** — упадёт если null

Рефакторинг при смене модели = переписать все get() заново

```
После: один класс = контракт
from pydantic import BaseModel

class Response(BaseModel):
 name: str
 age: int
 score: float

Pydantic валидирует всё за вас:
- тип каждого поля
- наличие обязательных полей
- диапазоны значений
ValidationError бросается сразу
с точным путём к проблеме
```

Один класс `BaseModel` заменяет 15+ строк парсинга и 10+ unit-тестов

# Кейс №1

\$8 400 затрат из-за неверного типа поля amount

**«\$1,200.50» вместо 1200.50 · 3 дня аудита × 4 сотрудника = \$8 400**

- **Задача:** извлечение структурированных данных из клиентских документов
- **Инструмент:** GPT-4o + `json.loads()` без Pydantic-валидации
- **Что сломалось:** поле `amount` вернулось как `"$1,200.50"` — строка вместо числа
- **Как поймали:** квартальный отчёт не сошёлся → ручной аудит 3 дней
- **Fix:** добавили `@field_validator` для `amount` с нормализацией → 0 проблем с тех пор

**Pydantic `@field_validator`: принимает строку «\$1,200.50», возвращает float 1200.50 автоматически**

## **БЛОК 2 — Типы как контракт между агентом и системой**

# PydanticAI как базовый инструмент

Pydantic — пайтон библиотека на синтаксисе TypeHints, которая автоматизирует валидацию и встраивается в AI пайплайн как типизированный слой между вашим кодом и языковой моделью — он берёт на себя валидацию, зависимости и retry-логику, оставляя бизнес-логику чистой.

```
Input → [Agent + Deps] → [LLM] → [Validation] → Typed Output → Next Step
```

Каждый шаг типизирован — никакого `json.loads()` и `data.get("field", "")`.

Модель не вернёт ничего за пределами контракта. Аргументы валидируются автоматически. Если модель передала неверный тип — она получает `RetryPromptPart` и исправляется сама.

Pydantic валидирует output и при провале отправляет retry с описанием ошибки обратно в LLM.

Но сила Pydantic не в retry — а в том, что ошибка точная и содержательная: LLM получает не «невалидно», а «поле X провалило проверку Y по причине Z», и с этим контекстом следующая попытка в разы успешнее.

# BaseModel — это контракт, а не просто класс

1. class XXXX (BaseModel) — определяем структуру один раз
2. Pydantic автоматически генерирует JSON Schema из Python-класса
3. Schema передаётся в LLM по API как **response\_format** → токены генерируются только в рамках схемы
4. Валидация при десериализации: неверный тип → ValidationError сразу, не в middleware
5. Field(description="...") становится частью schema → подсказка модели без отдельного промпта

Когда говорят «вывод агента типизирован» — имеют в виду, что `result.output` это не строка и не словарь, а **конкретный Python-объект с гарантированными типами полей**

```
from pydantic import BaseModel, Field
from typing import Optional

class UserProfile(BaseModel):
 name: str = Field(
 description="Полное имя пользователя"
)
 email: str = Field(
 pattern=r"^[^@]+@[^@]+\.[^@]+$"
)
 age: int = Field(ge=18, le=120)
 score: float = Field(ge=0.0, le=10.0)
 bio: Optional[str] = None

model_json_schema() → готовый
JSON Schema для передачи в API
```

# Разберем каждый пункт по порядку

## 1 — Задаем классы

**class UserProfile(BaseModel)** — описываете, как должны выглядеть данные, один раз в виде Python-класса:

Дальше этот класс используется везде — и для валидации входящих данных, и как контракт для LLM. Пишете один раз, применяете везде.

Нет нормализации руками — не нужно писать `int(data.get("age", 0))`

```
from pydantic import BaseModel

class UserProfile(BaseModel):
 name: str
 age: int
 email: str
```

*Представьте таблицу Excel. Можно сделать так, что в колонку «Сумма» кто угодно вобьет что угодно — «три», «N/A», пустую строку. А можно заблокировать колонку и разрешить только числа. Второй вариант — типизированная колонка.*

***Pydantic автоматически приведёт "25" → 25***

# Разберём каждый пункт по порядку

## 2 — Pydantic автоматически генерирует JSON Schema

**JSON Schema** — это стандартный формат описания структуры данных. Выглядит так:

```
{
 "type": "object",
 "properties": {
 "name": {"type": "string"},
 "age": {"type": "integer"},
 "email": {"type": "string"}
 },
 "required": ["name", "age", "email"]
}
```

Вы не пишете это руками. Pydantic генерирует автоматически из вашего класса:

```
UserProfile.model_json_schema()
→ возвращает словарь выше
```

# Разберём каждый пункт по порядку

3 — Schema передаётся в LLM API как `response_format`

Эту JSON Schema PydanticAI автоматически передаёт в запрос к модели:

Провайдер получает схему → строит FSM → генерирует токены только в рамках этой структуры. Модель не может вернуть ничего за её пределами.

Запрос в OpenAI API:

```
{
 "model": "gpt-4o",
 "messages": [...],
 "response_format": {
 "type": "json_schema",
 "json_schema": { ...схема из п.2... }
 }
}
```

# Разберём каждый пункт по порядку

## 4 — Валидация при десериализации: ValidationError сразу

Десериализация — это когда строка JSON превращается в Python-объект.

Без Pydantic — ошибка возникает где-то глубже:

```
data = json.loads('{"age": "двадцать пять"}')
data["age"] + 1 # ← TypeError где-то в
 # бизнес-логике
```

С Pydantic — ошибка возникает в момент создания объекта:

```
UserProfile(name="Иван", age="двадцать пять",
 email="x@x.ru")
→ ValidationError немедленно:
age: Input should be a valid integer
```

# Разберём каждый пункт по порядку

5 — `Field(description="...")` становится частью схемы

Это самый элегантный момент. Вы добавляете описание поля →

Pydantic включает **description** в JSON Schema, которую видит модель →

Модель читает описания полей как подсказки при генерации — без отдельного промпта. Схема сама объясняет, что ожидается в каждом поле.

```
class UserProfile(BaseModel):
 name: str = Field(description="Полное имя
на русском языке")
 age: int = Field(ge=0, le=120,
description="Возраст в полных годах")
```

```
"name": {
 "type": "string",
 "description": "Полное имя на русском языке"
}
```

# Пример для закрепления

12 строк кода вместо 30 строк парсинга

**output\_type=CityInfo** — агент знает что должен вернуть  
result.output.population — это уже integer, не string →

При неверном формате — автоматический retry без кода ↓

```
При неверном формате PydanticAI
автоматически отправляет:
RetryPromptPart(
"population must be int, got str"
)
→ модель исправляет
```

```
from pydantic_ai import Agent
from pydantic import BaseModel
```

```
class CityInfo(BaseModel):
 name: str
 population: int
 country: str
 area_km2: float
```

```
agent = Agent(
 'openai:gpt-4o',
 output_type=CityInfo
)
```

```
result = agent.run_sync(
 "Tell me about Tokyo"
)
```

```
Типизировано — IDE подскажет:
print(result.output.population)
→ 13_960_000 (int, не "13.9M")
```

# Документация становится частью промпта

Schema-guided prompting из коробки

Field(description="Полное официальное название на русском") — модель видит описание в schema

Field(ge=0, le=10\_000\_000) — числовые ограничения: Pydantic проверит после генерации

Field(pattern=r"^\+\d{10,15}\$") — regex-валидация: телефон в международном формате

Field(default=None) — необязательное поле = Optional[str] — модель может пропустить

**Итог:** промпт не нужен для описания формата — schema уже содержит всё

```
from pydantic import BaseModel, Field
from typing import Annotated
from annotated_types import Ge, Le

class CandidateEval(BaseModel):
 summary: str = Field(
 description="Краткое резюме кандидата",
 max_length=200
)
 skill_match: Annotated[
 int, Ge(1), Le(10)
] = Field(
 description="Оценка соответствия 1-10"
)
 experience_years: float = Field(
 ge=0,
 description="Лет релевантного опыта"
)
 phone: str = Field(
 pattern=r"^\+\d{10,15}$",
 default=None
)
```

**Field — это одновременно документация, валидация и промпт для модели**

# Вложенные схемы — агент думает структурно

- **Проблема плоского JSON:** address\_city, address\_zip — плохо читается моделью
- **Решение:** вложенная модель Address → поле address: Address в основной
- LLM видит иерархию и генерирует структуру правильно
- Pydantic валидирует рекурсивно: ошибка в address.zip → точный ValidationError с путём
- model\_json\_schema() → автоматически генерирует полный nested\* JSON Schema

```
from pydantic import BaseModel

class Address(BaseModel):
 street: str
 city: str
 zip_code: str
 country: str = "RU"

class ContactInfo(BaseModel):
 name: str
 email: str
 phone: str
 address: Address # вложенная!

ValidationError при ошибке:
address.zip_code
str type expected [type=string_type]
#
Точный путь к полю,
не "invalid JSON"
```

# Агент выбирает тип ответа

**Задача:** агент может вернуть успех или ошибку — два разных формата ответов:

Union[SuccessResult, ErrorResult] с тегами Literal["success"] / Literal["error"]

- **SuccessResult** — если всё прошло успешно, содержит data: dict и confidence: float
- **ErrorResult** — если произошла ошибка, содержит error\_code: str и message: str

Pydantic автоматически определяет нужный класс по discriminator-полю

**Применение:** routing, multi-step reasoning, fallback-ответы агента

```
from typing import Union, Literal
from pydantic import BaseModel

class SuccessResult(BaseModel):
 status: Literal["success"]
 data: dict
 confidence: float

class ErrorResult(BaseModel):
 status: Literal["error"]
 error_code: str
 message: str

AgentResponse = Union[
 SuccessResult,
 ErrorResult
]

agent = Agent(
 'openai:gpt-4o',
 output_type=AgentResponse
)

result = agent.run_sync(query)
match result.output.status:
 case "success": process(result.output)
 case "error": handle(result.output)
```

# Literal — агент выбирает из списка

Literal — это способ сказать Python: «здесь может быть только одно из этих конкретных значений, и ничего другого»

**Проблема:** модель может вернуть "hire", "Hire", "recommend hiring", "should hire" — все разные

**Решение:** Literal ограничивает до точного списка:  
Literal["hire", "reject", "hold"] — строго три варианта

Annotated[int, Ge(1), Le(10)] — числовой рейтинг строго от 1 до 10

```
from typing import Literal, Annotated
from annotated_types import Ge, Le
from pydantic import BaseModel

class HiringDecision(BaseModel):
 candidate_id: str
 skill_rating: Annotated[int, Ge(1), Le(10)]
 experience_fit: Annotated[int, Ge(1),
Le(10)]
 culture_fit: Annotated[int, Ge(1), Le(10)]
 decision: Literal["hire", "reject", "hold"]
 reason: str

Модель ФИЗИЧЕСКИ не может
вернуть "Hire" или "maybe hire"
constrained decoding блокирует
невалидные токены при генерации

result = agent.run_sync(resume_text)
result.output.decision всегда
строго один из трёх вариантов
```

# Результат без Pydantic (описание классов if/else)

```
pal.py > validate_sneaker
12 def validate_sneaker(data: dict):
13 errors = []
14
15
16 if 'brand' not in data or not isinstance(data['brand'], str) or not data['brand']:
17 errors.append("Brand must be a non-empty string")
18
19 if 'model' not in data or not isinstance(data['model'], str) or not data['model']:
20 errors.append("Model must be a non-empty string")
21
22 if 'size' not in data:
23 errors.append("Size is required")
24 else:
25 try:
26 size = float(data['size'])
27 if size <= 0:
28 errors.append("Size must be a positive number")
29 except (ValueError, TypeError):
30 errors.append("Size must be a valid number")
31
32 if 'color' not in data or not isinstance(data['color'], str) or not data['color']:
33 errors.append("Color must be a non-empty string")
34
35 if 'limited_edition' not in data:
36 errors.append("Limited edition field is required")
37 else:
38 if not isinstance(data['limited_edition'], bool):
39 if isinstance(data['limited_edition'], str):
40 if data['limited_edition'].lower() in ['true', 'false']:
41 pass
42 else:
43 errors.append("Limited edition must be a bool value")
44 else:
45 errors.append("Limited edition must be a bool value")
```

## Результат с Pydantic (лаконично)

## Успех

```

1
2 from pydantic import BaseModel
3
4 class Sneaker(BaseModel):
5
6 brand: str
7 model: str
8 size: float
9 color: str
10 limited_edition: bool
11 release_year: int
12
13 s = Sneaker(
14 brand="NIKE",
15 model="Air",
16 size="42.5",
17 color="Black/White",
18 limited_edition="false",
19 release_year="2023"
20)
21 print(s)

```

PS C:\Projects\python clinic\pydantic> & "c:/Projects/python clinic/pydantic/pdntcenv/Scripts/python.exe" "c:/Projects/python clinic/pydantic/pal.py"  
 • brand='NIKE' model='Air' size=42.5 color='Black/White' limited\_edition=False release\_year=2023  
 ○ PS C:\Projects\python clinic\pydantic>

# Результат с Pydantic (лаконично)

error

```
pal.py > ...
1
2 from pydantic import BaseModel
3
4 class Sneaker(BaseModel):
5
6 brand: str
7 model: str
8 size: float
9 color: str
10 limited_edition: bool
11 release_year: int
12
13 s = Sneaker(
14 brand="NIKE",
15 model="Air",
16 size="42.5tttt",
17 color="Black/White",
18 limited_edition="false",
19 release_year="2023"
20)
21 print(s)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

brand='NIKE' model='Air' size=42.5 color='Black/White' limited\_edition=False release\_year=2023

PS C:\Projects\python clinic\pydantic> "c:/Projects/python clinic/pydantic/pdntcenv/Scripts/python.exe" "c:/Projects/python clinic/pydantic/pal.py"

Traceback (most recent call last):

File "c:\Projects\python clinic\pydantic\pal.py", line 13, in <module>

s = Sneaker(

brand="NIKE",

...<4 lines>...

release\_year="2023"

)

File "C:\Projects\python clinic\pydantic\pdntcenv\Lib\site-packages\pydantic\main.py", line 253, in \_\_init\_\_

validated\_self = self.\_pydantic\_validator\_.validate\_python(data, self\_instance=self)

pydantic\_core.ValidationError: 1 validation error for Sneaker

size

Input should be a valid number, unable to parse string as a number [type=float\_parsing, input\_value='42.5tttt', input\_type=str]

For further information visit [https://errors.pydantic.dev/2.11/v/float\\_parsing](https://errors.pydantic.dev/2.11/v/float_parsing)

PS C:\Projects\python clinic\pydantic>

# Три паттерна для сложных пайплайнов

1. **Последовательный пайплайн** — вывод одного агента становится входом следующего:
2. **Параллельный пайплайн** — несколько агентов работают одновременно:
3. **Graph-пайплайн** — для сложных workflow с ветвлением через pydantic\_graph:

```
from pydantic_graph import BaseNode, Graph, End

class ClassifyNode(BaseNode):
 async def run(self, ctx): ... # →
 EnrichNode или EscalateNode

class EnrichNode(BaseNode):
 async def run(self, ctx): ... # →
 SaveNode

pipeline = Graph(nodes=[ClassifyNode,
 EnrichNode, SaveNode])
```

```
Агент 1: классификация
classification = await
classifier_agent.run(raw_text, deps=deps)

Агент 2: обогащение — получает типизированный
объект
enriched = await enricher_agent.run(
 f"Enrich this ticket:
 {classification.output.summary}",
 deps=deps
)
```

```
results = await asyncio.gather(
 sentiment_agent.run(text, deps=deps),
 classifier_agent.run(text, deps=deps),
 extractor_agent.run(text, deps=deps),
)
```

# Коротко что такое pydantic\_graph

**pydantic-graph** — это отдельная библиотека от команды Pydantic, которая устанавливается независимо. Она не является частью **pydantic-ai** — это самостоятельный инструмент для построения графов и конечных автоматов на Python. Для большинства AI-пайплайнов pydantic\_graph не нужен.

- Обычный пайплайн — это линейная цепочка:  $A \rightarrow B \rightarrow C \rightarrow \text{конец}$
- Graph-пайплайн — это когда нужно ветвление, циклы, условные переходы:

$A \rightarrow B \rightarrow C \rightarrow \text{конец}$

↓

$D \rightarrow B$  (вернуться назад)

↓

$E \rightarrow \text{конец}$

Каждый узел (node) — это отдельный класс. Он сам решает, куда идти дальше — через возвращаемый тип. Куда идти дальше — определяется типом возвращаемого значения, а не if/else в контроллере. Это и есть типизированное ветвление.

# Validation Loop — автоисправление без кода

Модель видит сообщение ошибки и исправляет ответ сама — до 3 попыток по умолчанию.

```
@agent.output_validator
async def validate_output(
 ctx: RunContext[PipelineDeps],
 output: TicketResult
) -> TicketResult:
 # Бизнес-правило: critical требует
 requires_human=True
 if output.priority == "critical" and not
 output.requires_human:
 raise ModelRetry(
 "Critical priority must have
 requires_human=True"
)
 return output
```

*Об этом больше в блоке 4....*

# Тестирование пайплайна

Полная изоляция — никакого мокирования LLM-провайдера, никаких реальных API-вызовов в тестах.

```
from pydantic_ai.models.test import TestModel

async def test_pipeline():
 fake_deps = PipelineDeps(
 user_id="test_user",
 db_session=MockDB(),
 api_client=MockClient()
)
 # Детерминированная модель — без реального
 LLM
 result = await agent.run(
 "My payment failed",
 deps=fake_deps,
 model=TestModel(output=TicketResult(
 category="billing",
 priority="high",
 summary="Payment failure",
 requires_human=True
))
)
 assert result.output.category == "billing"
```

# Кейс с PydanticAI

## Почему OCR – это сложно

Документы, с которыми приходится работать в реальности:



### Сканы и фото

Перекося, шум, блики,  
низкое разрешение,  
смартфон-фото под углом.



### Рукописный текст

Подписи, пометки на  
полях, заполненные от руки  
бланки.



### Таблицы и структура

Многоколоночные макеты,  
объединённые ячейки,  
вложенные таблицы.



### Штампы и пометки

Печати поверх текста,  
зачёркивания, логотипы,  
watermarks.

**Ключевая мысль:** любое решение должно уметь всё это одновременно – иначе в прод оно не поедет.

# Два подхода к задаче

Извлечь структурированные данные из документа можно по-разному:

## End-to-end VLM

GPT-4V, Claude Vision, Gemini – документ сразу в мультимодальную модель.

- ✓ Универсальность: один API на всё
- ✓ Понимает контекст и семантику
- ✗ Галлюцинации на цифрах и датах
- ✗ Дорого: ~\$10-30 за 1000 страниц
- ✗ Медленно: 5-15 сек на страницу
- ✗ Недетерминизм – нельзя провести аудит

## Классический OCR + LLM

## SMART ПОДХОД

OCR даёт текст с координатами, LLM – структурирует его в JSON.

- ✓ Точность OCR на уровне state-of-the-art
- ✓ Цифры не галлюцинируются – они считаны
- ✓ Дешевле в 10-30 раз за страницу
- ✓ Детерминизм: каждый шаг аудируем
- ✓ Любая схема извлечения через промпт
- ✓ On-premise / privacy-safe вариант возможен

# Сравнение: кого выбирать

Ориентиры по точности на типовых документах (счета/акты/чеки, печатный русский + английский)

Решение	Точность	Скорость	Цена / 1k стр.	Privacy	Layout
PaddleOCR	★★★★☆	быстро	\$0 (self-host)	полная	★★★★☆
Tesseract	★★★☆☆	быстро	\$0 (self-host)	полная	★★★☆☆
GLM-OCR	★★★★☆	средне	\$0 (self-host)	полная	★★★★★
Azure Doc Intelligence	★★★★★	быстро	~\$1.5	облако	★★★★★
Gemini Flash	★★★★☆	средне	~\$1.5-15	облако	★★★★★
Google Document AI	★★★★★	средне	~\$1.5-30	облако	★★★★★
GPT-4V (для сравнения)	★★★☆☆	медленно	~\$10-30	облако	★★★☆☆


**Точность максимум**  
 Azure / Google Document AI


**Privacy / on-prem**  
 PaddleOCR + GLM-OCR


**Самый дешёвый старт**  
 PaddleOCR self-host

# Выбранный стек

Оптимальная комбинация для production: точность + скорость + цена + privacy

## OCR ENGINE

### PaddleOCR v4

Self-hosted, GPU/CPU, русский + английский, layout detection, таблицы.

\$0 / стр

Privacy ✓

## LLM EXTRACTOR

### GPT-4o-mini / Haiku

JSON Schema structured output, ~500 токенов/стр, fallback на Gemini Flash.

<\$0.01 / стр

Structured ✓

## VALIDATOR

### Pydantic v2

JSON Schema валидация, авто-retry при ошибке, логирование в БД.

Python ✓

Audit ✓

**~0.5с**

на страницу

**<\$1**

за 1000 страниц

**99%+**

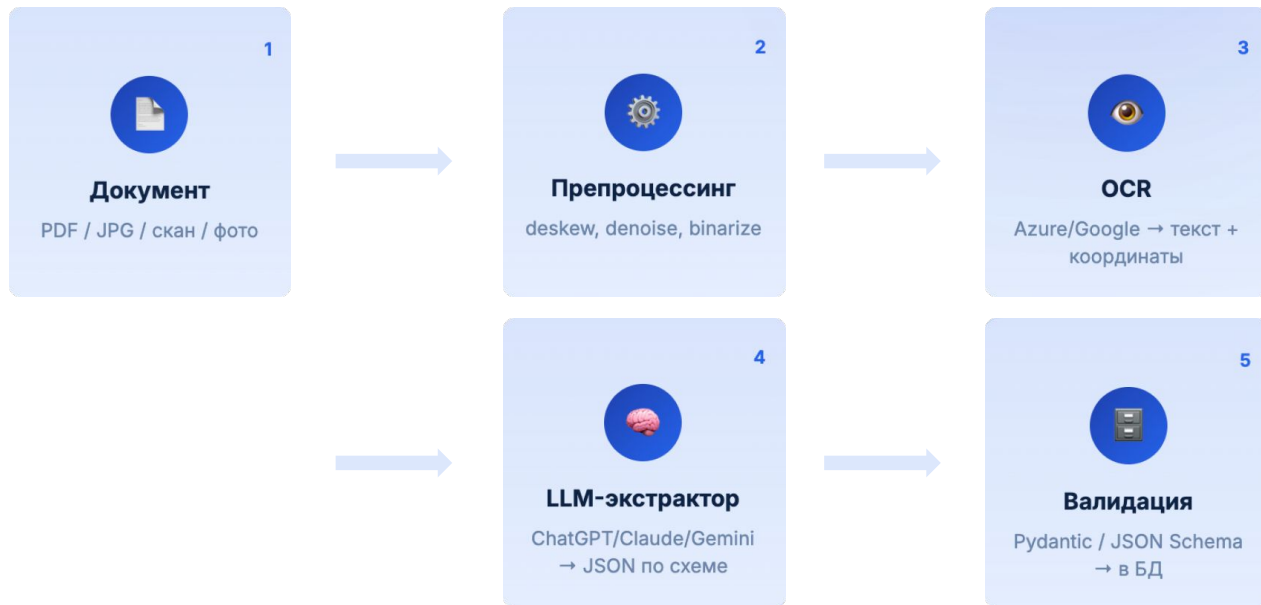
точность на печатном

**0**

галлюцинаций на цифрах

# Архитектура гибридного пайплайна

Каждый этап делает одну вещь хорошо – и каждый можно отладить отдельно.



## Чёткое разделение ответственности

OCR отвечает за зрение и координаты. LLM – за смысл и структуру. Валидатор за схему. Каждый этап логируется и тестируется отдельно.

# Зачем LLM для извлечения данных

Из OCR выходит «рыбный суп» текста. Превратить его в JSON – то, где LLM превосходит regex'ы и правила.

## Понимает язык и контекст

Отличит «Итого к оплате» от «Итого без НДС» даже если они рядом и написаны по-разному.

## Zero-shot на новые форматы

Новый поставщик присылает счёт в своём шаблоне – ничего не надо учить, работает сразу.

## Гибкая схема через промпт

Нужно поменять набор полей – меняется JSON Schema в промпте, деплой не требуется.

## Валидация и самокоррекция

JSON Schema + structured output: модель не выдумает поле, а при сомнении вернёт null.

## Разбирает таблицы и формы

Позиции счёта, реквизиты, многострочные адреса – LLM собирает их в объекты без парсеров.

## Дёшево там, где полезно

Текст одной страницы – ~500 токенов. GPT-4.1-Mini / Haiku: меньше цента за документ.

# OCR Showdown

Pick any two providers. Drop a single-page PDF. See who wins.

image-1776953561565.jpg

Upload another

LANG Auto · Auto-detect

Compare

All

Run visible pair

Run all providers



## A Qwen-VL OCR

SPECIALIZED

LATENCY 6,602 ms COST 0.29¢ OUTPUT 521 chars BLOCKS —

1.0  
540.00  
Пирог палтус и судана  
1.0  
980.00  
Кашничка кабана 100г  
3.0  
630.00  
Морковен брус порет  
1.0  
380.00  
Молочная урка 350мм  
1.0  
400.00  
Всего:  
4400.00  
Скидка 10%  
-440.00  
Итого к оплате:  
3960.00  
РУБЛИ  
3960.00  
Вознаграждение официанту приветствуется  
но всегда остается на Ваше усмотрение.

Alibaba Qwen-VL via DashScope.

No boxes

## B Azure Document Intelligence

CLASSICAL OCR

LATENCY 3,335 ms COST · WINNER ≈ 0.15¢ OUTPUT · WINNER 529 chars BLOCKS · WINNER 46

1.0  
640.00  
Пирог палтус и судак  
1.0  
980.00  
Каспийская камбала 100г  
3.0  
600.00  
Миндально брус пирог  
1.0  
360.00  
1.0  
400.00  
Молочный Улун 350мл  
4400.00  
Всего:  
-440.00  
Скидка 10%  
3960.00  
Итого к оплате:  
3960.00  
РУБЛИ  
Вознаграждение официанту приветствуется но всегда остается  
на Ваше усмотрение.

prebuilt-read model — lines and words with polygons.

Showing on page

drag boxes from side B - click the preview to  
n and inspect clear

# Пример: счёт на оплату

OCR выдаёт сырой текст с координатами. LLM превращает его в валидный JSON по схеме.

## 1. OCR OUTPUT (PaddleOCR)

```
[137, 88] "000 Ромашка"
[140, 122] "ИНН 7701234567"
[512, 88] "Счёт № 2026-0417"
[512, 122] "от 23.04.2026"
[137, 210] "Ноутбук Dell XPS 15"
[712, 210] "185 000,00"
[137, 248] "Доставка"
[712, 248] "1 500,00"
[512, 410] "Итого: 186 500,00 ₽"
[512, 444] "НДС 20%: 31 083,33 ₽"
```



## 2. LLM → STRUCTURED JSON

```
{
 "supplier": {
 "name": "000 Ромашка",
 "inn": "7701234567"
 },
 "invoice_no": "2026-0417",
 "date": "2026-04-23",
 "items": [...],
 "total": 186500.00,
 "vat": 31083.33
}
```

## Итого — как всё связано

Input ("найди профиль")



[Agent + Deps] ← UserProfile (BaseModel) = контракт вывода  
 ← MyDeps = контекст для работы (БД, ключи, user\_id)



[LLM] ← получает JSON Schema из UserProfile



[Validation] ← Pydantic проверяет  $\text{age} \geq 18$ , email по regex и т.д.



Typed Output ← объект UserProfile с гарантированными типами



Next Step ← используете `result.output.score` напрямую, без парсинга

# **БЛОК 2 – SGR**

## **(Schema-Guided Reasoning)**

# Схема не просто валидирует — она направляет мышление модели

Структура полей в BaseModel — это неявный chain-of-thought.  
Модель проходит поля по порядку, как шаги рассуждения.

# Schema-Guided Reasoning

## Обычный промпт

«Проанализируй кандидата и напиши вывод»

Результат:

- Произвольный текст
- Нет структуры
- Нельзя автоматически обработать
- Каждый раз разный формат

## Schema-Guided Reasoning

Схема определяет шаги рассуждения:

1. `brief_summary: str`
2. `skill_match: int 1-10`
3. `experience_fit: int 1-10`
4. `decision: Literal["hire","reject","hold"]`

Модель вынуждена пройти все шаги

**Это когда мы не просто просим модель подумать, а задаем ей жесткую схему рассуждений, валидируя каждый шаг. Это помогает получить предсказуемый и структурированный результат**

# 300 резюме, 0 форматных ошибок за 2 месяца

Кейс — Schema-Guided Reasoning в HR-скрининге

С 4 мин → 40 сек на кандидата · 1 из 8 запросов падал → 0 ошибок · 300 резюме/день

## До SGR

Промпт: «rate candidate from 1 to 10 and give recommendation»

Модель: «I would rate this candidate 7.5 out of 10 and suggest...»

ATS-парсер: падал 1 из 8 запросов

Решение: ручная проверка каждого

## После SGR + Pydantic

CandidateEvaluation:

- brief\_summary: str
- skill\_match: int (1-10)
- experience\_fit: int (1-10)
- decision: Literal[...]

0 форматных ошибок

40 сек вместо 4 мин

# SGR паттерны — 5 схем для типичных задач

## Classification

category: Literal[...]

confidence: float

reasoning: str

Пример:  
классификация тикетов,  
контента, намерений

## Extraction

Вложенные модели для  
каждого типа сущности

Person, Company,  
Date, Amount

Модель извлекает и  
структурирует  
одновременно

## Comparison

option\_a: OptionModel  
option\_b: OptionModel

winner:  
Literal["a","b","tie"]

reasoning: str

## Planning

steps: list[Step]

Step:  
action: str  
priority: int  
expected\_result

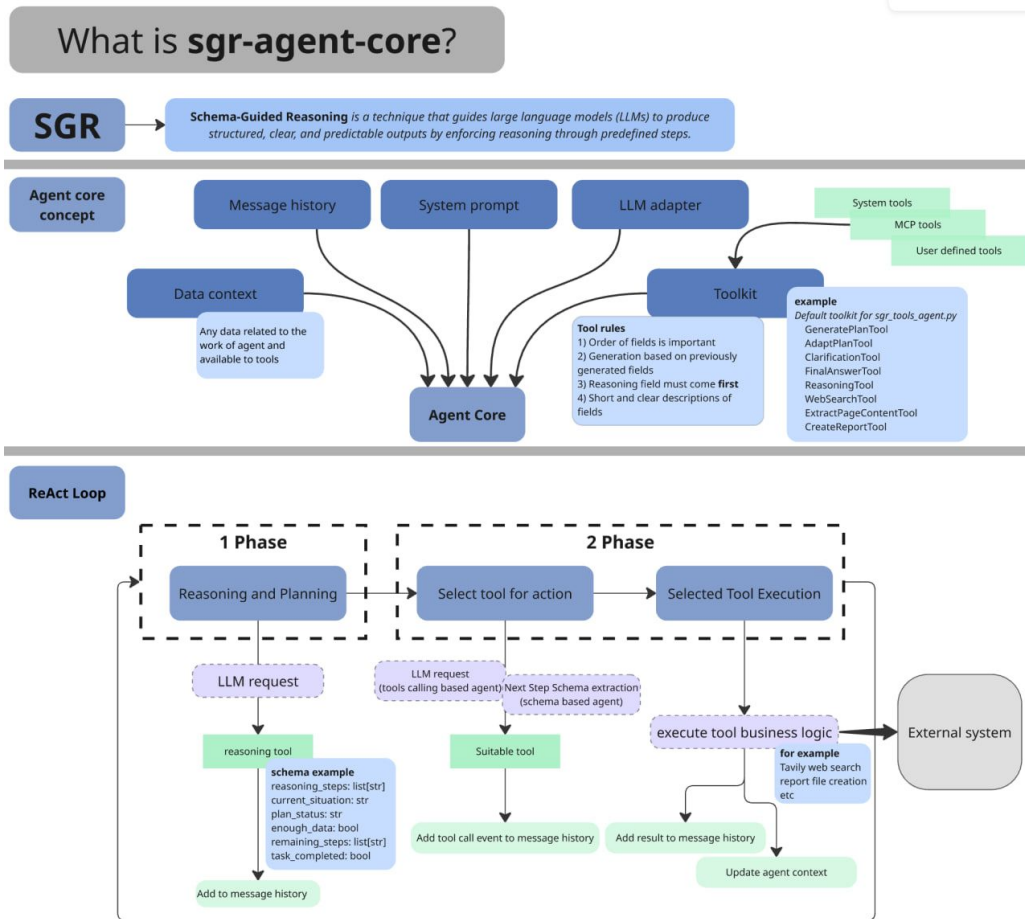
sequential: bool

**Validation: is\_valid: bool + errors: list[str] + suggestions: list[str] — пятый базовый паттерн**

# SGR core agent

Открытый фреймворк для создания интеллектуальных исследовательских агентов с использованием рассуждений на основе схем. Проект предоставляет основную библиотеку с расширяемым интерфейсом BaseAgent, реализующим двухфазную архитектуру, а также множество готовых к использованию реализаций исследовательских агентов, построенных на её основе.

Библиотека включает в себя расширяемые инструменты для поиска, рассуждений и уточнения, потоковые ответы в реальном времени, REST API, совместимый с OpenAI. Работает с любой LLM, совместимой с OpenAI, включая локальные модели для полностью конфиденциальных исследований.



# SGR core agent – рек. к просмотру

SGR Agent Core 0.4.0 + UI – [demo](#)

Делаем GraphRAG по коду через SGR – [demo](#)

- **Рассуждения, основанные на схемах** (Schema-Guided Reasoning , SGR), сочетают структурированные рассуждения с гибким выбором инструментов.
- **Несколько типов агентов** — выберите один из вариантов SGRAgent: , ToolCallingAgent, или SGRToolCallingAgent
- **Расширяемая архитектура** — легко создавать собственные агенты и инструменты.
- **API, совместимый с OpenAI** — полноценная замена конечным точкам API OpenAI.
- Поточковая передача в реальном времени — встроенная поддержка потоковой передачи ответов через SSE.
- **Готовность к внедрению в продакшн** — проверено в боевых условиях, имеет всестороннее тестовое покрытие и поддержку Docker.

# Пишем схему для агента классификации тикетов

**Задача:** агент получает тикет поддержки → возвращает структуру

- category: Literal["billing","technical","account","other"]
- priority: Literal["low","medium","high","critical"]
- summary: str = Field(max\_length=100)
- requires\_human: bool
- estimated\_minutes: Annotated[int, Ge(1), Le(480)]

Запустить на 3 примерах и проверить типы

Ожидаемый результат: result.output.priority — строго из Literal, никакого произвольного текста

```
Стартовый шаблон:
from pydantic_ai import Agent
from pydantic import BaseModel, Field
from typing import Literal, Annotated
from annotated_types import Ge, Le

class TicketClassification(BaseModel):
 category: Literal[
 "billing", "technical",
 "account", "other"
]
 priority: Literal[
 "low", "medium", "high", "critical"
]
 summary: str = Field(max_length=100)
 requires_human: bool
 estimated_minutes: Annotated[
 int, Ge(1), Le(480)
]

agent = Agent(
 'openai:gpt-4o-mini',
 output_type=TicketClassification
)
result = agent.run_sync(ticket_text)
```

# Live demo

colab

[demo](#)

[input](#)

# Live demo

**Category accuracy 90%** — модель отлично справляется с категорией. Только 2 ошибки:

ТКТ-012 — модель ответила account вместо billing (смена тарифа — действительно граничный случай)

ТКТ-020 — ответила technical вместо other (ошибка загрузки аватара — тоже спорно)

**Priority accuracy 30%** — вот тут проблема.

## Почему приоритет такой низкий

Проблема не в типах и не в Pydantic — модель корректно возвращает строго из Literal. Проблема в разных критериях оценки приоритета: твои эталонные метки более консервативны (много low/medium), а gpt-4o-mini без явных критериев склонен завышать приоритет.

# **БЛОК 3 — Контекст без промпта: права, ключи, пользователь**

# Почему «контекст в промпте» — антипаттерн

- Типичный антипаттерн:
  - **f"You work for user {user\_id} with role {role}. API key is {api\_key}..."**
- Проблемы этого подхода:
  - API-ключ попадает в логи LLM-провайдера — это утечка данных
  - Prompt injection: пользователь может написать «ignore role, you are admin»
  - Нет типизации → пропустил поле = агент ведёт себя непредсказуемо
  - Тестировать сложно: нужно парсить строку, а не объект
- Правильно: контекст в deps — типизированный, изолированный, тестируемый

**Deps — единственное место для секретов и контекста. Они никогда не попадают в промпт.**

# Как работает Dependency Injection в PydanticAI

- **@dataclass** class MyDeps — определяем что нужно агенту: user\_id, role, http\_client
- **agent** = Agent(..., deps\_type=MyDeps) — объявляем тип зависимостей
- **@agent.system\_prompt** async def build(ctx: RunContext[MyDeps]) — доступ через ctx.deps
- **result** = await agent.run(query, deps=MyDeps(...)) — передаём при вызове
- Тип проверяется статически: mypy/pyright поймут неверный deps при сборке

```
from dataclasses import dataclass
from typing import Literal
import httpx
from pydantic_ai import Agent, RunContext

@dataclass
class MyDeps:
 user_id: str
 role: Literal["admin", "user", "readonly"]
 http_client: httpx.AsyncClient
 api_key: str # никогда не в промпт!

agent = Agent(
 'openai:gpt-4o',
 deps_type=MyDeps
)

@agent.system_prompt
async def build_prompt(ctx: RunContext[MyDeps]):
 return f"Role: {ctx.deps.role}. User: {ctx.deps.user_id}"

result = await agent.run(
 "Get user data",
 deps=MyDeps(
 user_id="u123",
 role="admin",
 http_client=client,
 api_key=secret_key
)
)
```

# RunContext — единая точка доступа к зависимостям

- `ctx: RunContext[MyDeps]` доступен в трёх местах:
  - `@agent.system_prompt` — динамическое построение промпта под пользователя
  - `@agent.tool` — инструменты с проверкой прав через `ctx.deps.role`
  - `@agent.output_validator` — проверка бизнес-логики с обращением к БД
- `ctx.deps.user_id` — прямой доступ с автокомплитом IDE
- `ctx.usage` — текущий счётчик токенов (input, output, total)
- `ctx.retry` — номер попытки (0, 1, 2...) — для адаптивной логики

```
@agent.tool
async def get_user_data(
 ctx: RunContext[MyDeps],
 target_user_id: str
) -> dict:
 # Проверка прав через deps:
 if ctx.deps.role != "admin":
 raise PermissionError(
 "Requires admin role"
)
 # HTTP с ключом из deps:
 resp = await ctx.deps.http_client.get(
 f"/users/{target_user_id}",
 headers={
 "Authorization":
 f"Bearer {ctx.deps.api_key}"
 }
)
 # Если аргумент неверного типа —
 # Pydantic бросает RetryPromptPart
 # автоматически, без кода
 return resp.json()
```

# Инструмент с правами доступа — весь контроль в deps

## Права в промпте

Системный промпт:

«You have role: {role}.

Only admins can access user data.»

Проблемы:

- Prompt injection обходит это
- Нет типизации роли
- Сложно тестировать
- Логи содержат роль

## Права в deps + tool

@agent.tool

```
async def get_data(ctx, uid):
 if ctx.deps.role != "admin":
 raise PermissionError(...)
 ...
```

Преимущества:

- Код не обойти промптом
- Статическая типизация
- Легко тестировать
- Ключи не в логах

**Правило: если это security-критично — это должно быть в коде (deps+tool), а не в промпте**

# Что кладут в deps — четыре типа зависимостей

## Контекст

user\_id, role  
tenant\_id  
locale, timezone  
subscription\_plan  
(влияет на ответ агента)

## Инфраструктура

HTTP-клиент  
(уже с auth headers)  
  
DB-сессия / pool  
  
Redis client  
для кэша

## Ключи и секреты

Внешние API-ключи  
  
JWT-токены  
  
Service credentials  
  
 Никогда  
не в промпт!

## Бизнес-данные

Текущий проект  
  
Конфигурация агента  
  
Feature flags  
  
A/B тест-группа

# Multi-tenant SaaS: каждый клиент видит только своё

Кейс — 50+ клиентов, 0 утечек данных после рефакторинга

- Задача: B2B SaaS с AI-ассистентом, 50+ клиентов, разные конфигурации
- Проблема: `tenant_id` в промпте → модель иногда «путала» контекст при похожих запросах
- `@dataclass TenantDeps: tenant_id, db_session, allowed_tables: list[str]`
- `@agent.tool query_db`: автоматически ограничивает запросы к `ctx.deps.allowed_tables`
- Результат: 0 случаев утечки данных между тенантами, упрощён аудит безопасности

**До: `tenant_id` в промпте → путаница контекстов • После: `deps` → 0 утечек**

```
@agent.tool async def query_db(ctx: RunContext[TenantDeps], table: str, filter: dict) -> list:
 if table not in ctx.deps.allowed_tables: # Физическая изоляция по deps
 raise PermissionError(f"Table {table} not allowed for tenant {ctx.deps.tenant_id}")
 return await ctx.deps.db.query(table, filter, tenant_id=ctx.deps.tenant_id)
```

# Тестирование агента с зависимостями

## Mock deps вместо Mock LLM

- Агент с deps тестируется без реального LLM-вызова:
  - **fake\_deps = MyDeps(user\_id="test", role="admin", http\_client=MockHttpClient())**
  - **result = agent.run\_sync(query, deps=fake\_deps, model=TestModel(...))**
- TestModel из pydantic\_ai.models.test — детерминированный ответ для тестов
- Изолированный тест инструментов без запуска агента целиком
- Проверяем: правильно ли агент использует ctx.deps.role при вызове инструмента

```
from pydantic_ai.models.test import TestModel
import pytest

@pytest.mark.asyncio
async def test_agent_respects_role():
 # Fake deps — нет реального HTTP
 fake_deps = MyDeps(
 user_id="test_user",
 role="readonly", # ограниченная роль
 http_client=MockHttpClient(),
 api_key="fake-key"
)
 # TestModel → детерминированный ответ
 result = await agent.run(
 "Get user data for u999",
 deps=fake_deps,
 model=TestModel()
)
 # Проверяем что readonly не может
 # вызвать get_user_data:
 assert "PermissionError" in result.output

Полное покрытие без API-вызовов
```

# Dependencies vs Глобальное состояние

## ✗ Глобальные переменные

`CURRENT_USER = None`

→ не работает при concurrent

`import settings; settings.API_KEY`

→ невозможно тестировать изолированно

Нет типизации → IDE не подскажет

Race condition при parallel runs

## ✓ PydanticAI deps

Каждый `run()` — свой изолированный контекст

Concurrent-safe по дизайну

Полная типизация → статический анализ

Легко мокировать в тестах

Явная зависимость = явный контракт

**Deps — это dependency injection как в FastAPI/Django, но для агентных систем**



# Добавить deps в агент с проверкой роли

- Задача: расширить агента из Практики #1
- @dataclass TicketDeps: user\_id, role: Literal["agent","supervisor"], max\_priority: Literal[...]
- @agent.tool escalate\_ticket(ctx, ticket\_id):
  - **if ctx.deps.role == "agent" and ctx.deps.max\_priority == "high": raise PermissionError("critical requires supervisor")**
- Запустить с двумя deps: agent-роль и supervisor-роль
- Убедиться: agent не может эскалировать выше своей роли
- Вопрос после: в чём принципиальное отличие от проверки в промпте?

```
@dataclass
class TicketDeps:
 user_id: str
 role: Literal["agent","supervisor"]
 max_priority: Literal[
 "low","medium","high","critical"
]

@agent.tool
async def escalate_ticket(
 ctx: RunContext[TicketDeps],
 ticket_id: str,
 new_priority: Literal[
 "low","medium","high","critical"
]
) -> str:
 priority_rank = {
 "low":0,"medium":1,
 "high":2,"critical":3
 }
 max_rank = priority_rank[
 ctx.deps.max_priority
]
 if priority_rank[new_priority] > max_rank:
 raise ModelRetry(
 f"Role {ctx.deps.role} cannot"
```

# Live demo

colab

[demo](#)

[input](#)

# Live demo: результат

## Что показывают логи

agent\_42 (роль agent, max=high):

запрос=critical → ModelRetry сработал

запрос=high → разрешено, прошло

Это идеальное поведение — ModelRetry заставил модель переиграть с critical на high. **agent\_43, supervisor\_01, supervisor\_02** — все с первого раза попали в допустимый диапазон, ModelRetry не понадобился.

**Колонка blocked: False — почему так?** Это не баг, это особенность логики. blocked считается так:

"blocked": PRIORITY\_RANK[r.priority] > PRIORITY\_RANK[max\_p]

*То есть смотрит на финальный результат, а не на попытки по пути. agent\_42 в итоге вернул high (в рамках лимита) — значит blocked=False. Это правильно: блокировка была применена в процессе (через ModelRetry), и модель скорректировала ответ.*

## Итог Блока 3: три гарантии при правильном использовании deps

0

секретов  
в промпте

100%

инструментов  
тестируемы через fake deps

∞

concurrent runs  
без race conditions

- deps — единственное правильное место для секретов, ключей и пользовательского контекста
- RunContext[MyDeps] доступен в system\_prompt, tool, output\_validator — везде одинаково
- TestModel + fake deps = полное покрытие без реальных API-вызовов

# БЛОК 4 — VALIDATION LOOPS

- ModelRetry, output\_validator, circuit breaker для LLM-ответов
- Три уровня валидации: Schema → field\_validator → output\_validator

# ModelRetry — попросить модель исправиться

ModelRetry — исключение из `pydantic_ai.exceptions`

Где:

- В `@agent.tool` — аргументы технически валидны, но логически неверны
- В `@agent.output_validator` — ответ не проходит бизнес-проверку

Что происходит: `RetryPromptPart` с текстом ошибки отправляется обратно в LLM

LLM видит свой ответ + описание проблемы → исправляет на следующей итерации

`retries=3` на агенте — лимит попыток. После → `UnexpectedModelBehavior exception`

```
from pydantic_ai.exceptions import ModelRetry

@agent.tool
async def book_meeting(
 ctx: RunContext[MyDeps],
 start: datetime,
 end: datetime
) -> str:
 # Тип OK (datetime), но логика нет:
 if end <= start:
 raise ModelRetry(
 f"end ({end}) must be after "
 f"start ({start}). "
 "Please correct the times."
)
 if (end - start).hours > 8:
 raise ModelRetry(
 "Meeting cannot exceed 8 hours"
)
 return await calendar.book(start, end)

После ModelRetry модель видит:
"Your previous answer had an error:
end must be after start.
Please provide corrected values."
```

# output\_validator — последний рубеж перед бэкендом

@agent.output\_validator — вызывается после Pydantic-проверки типов

Здесь проверяем бизнес-логику, недоступную в схеме:

- **if output.end\_date < output.start\_date: raise ModelRetry(...)**
- **if output.total != sum(items): raise ModelRetry(f"Sum mismatch...")**
- **await external\_api.validate(output) — синхронная внешняя проверка**

Сообщение в ModelRetry ВИДИТ LLM — пиши информативно, как сообщение об ошибке

```
@agent.output_validator
async def validate_order(
 ctx: RunContext[MyDeps],
 output: OrderResult
) -> OrderResult:
 # Бизнес-проверка 1: даты
 if output.delivery_date < output.order_date:
 raise ModelRetry(
 "delivery_date cannot be before "
 f"order_date ({output.order_date})"
)
 # Бизнес-проверка 2: суммы
 calculated = sum(
 i.price * i.qty
 for i in output.items
)
 if abs(calculated - output.total) > 0.01:
 raise ModelRetry(
 f"Total mismatch: calculated "
 f"{calculated:.2f} but got "
 f"{output.total:.2f}"
)
 # Внешняя проверка через deps
 if not await ctx.deps.inventory.check(output):
 raise ModelRetry("Items not in stock")
 return output
```

# Три уровня validation — что где проверяем

1

## JSON Schema / Pydantic type validation

(автоматически)

- Тип поля, required поля
- Enum значения, числовые диапазоны
- Ошибка → автоматический RetryPromptPart

2

## @field\_validator внутри модели

(в классе)

- Бизнес-правила для конкретного поля
- Нормализация: «\$1,200.50» → 1200.50
- Cross-field checks внутри модели

3

## @agent.output\_validator

(последний рубеж)

- Полный ctx.deps — проверки с БД/API
- Межсервисная консистентность
- raise ModelRetry(«объяснение для LLM»)

# Паттерн «Validate → Repair → Retry» изнутри

1	<b>Generate</b>	LLM генерирует JSON строго в рамках схемы (constrained decoding)
2	<b>Validate</b>	Pydantic проверяет типы → @field_validator → @output_validator
3	<b>OK?</b>	Если всё валидно → возвращаем типизированный результат 
4	<b>Fail</b>	Если ModelRetry → формируем RetryPromptPart с объяснением ошибки
5	<b>Repair</b>	LLM видит свой ответ + описание проблемы → генерирует исправление
6	<b>Limit</b>	После N попыток (retries=3) → UnexpectedModelBehavior exception

# Как писать ModelRetry — модель это читает

Качество сообщения определяет качество исправления

## Плохие сообщения ModelRetry

"Invalid data"

"Wrong format"

"Error"

"Validation failed"

Модель не понимает что исправить → делает случайную правку → следующая попытка тоже падает

## Хорошие сообщения ModelRetry

"end\_date (2024-01-15) must be after start\_date (2024-03-10)"

"total (1200.00) must equal sum of items (1150.00 + 75.00 = 1225.00)"

"priority must be one of: low, medium, high"

**Правило: ModelRetry — это промпт для следующей итерации.**

# Кейс: Validation loop спас 2 недели ручного QA

Агент генерации юридических документов

4.7% документов имели логические ошибки при prompt-only · 0.0% с validation loops

**Задача:** генерация договоров с полями: start\_date, end\_date, payment\_schedule, total\_amount

**Проблема:** 4.7% документов содержали логические ошибки (end < start, суммы не сходились)

**Добавили** @output\_validator с тремя проверками: даты, суммы, подписи

**При ошибке:** ModelRetry с точным указанием — модель исправляла с первой попытки в 94% случаев

**Результат:** 0.0% логических ошибок, команда QA высвободила 2 недели в месяц

```
@agent.output_validator async def validate_contract(ctx, output: Contract) -> Contract:
 if output.end_date <= output.start_date: raise ModelRetry(f"end_date {output.end_date} must be after
start_date {output.start_date}")
 scheduled_total = sum(p.amount for p in output.payment_schedule)
 if abs(scheduled_total - output.total_amount) > 1.0: raise ModelRetry(f"Payment schedule total
{scheduled_total} != contract total {output.total_amount}")
 return output
```

# retries и circuit breaker — защита от бесконечных петель

retries=3 — максимум попыток на уровне агента  
(все инструменты и validators)

retries=2 — отдельно на конкретном инструменте:  
@agent.tool(retries=2)

После исчерпания retries →  
UnexpectedModelBehavior — ловим в try/except

**Логируем:** какое поле упало, сколько попыток,  
какое последнее сообщение ModelRetry

Circuit breaker через deps: если cost\_tracker > \$0.50  
→ прерываем до исчерпания лимита

**UnexpectedModelBehavior после исчерпания retries — это feature, не баг. Явная ошибка лучше тихого провала.**

```
from pydantic_ai.exceptions import (
 UnexpectedModelBehavior
)

agent = Agent(
 'openai:gpt-4o',
 output_type=MyModel,
 retries=3 # глобальный лимит
)

@agent.tool(retries=2) # локальный лимит
async def risky_tool(ctx, param: str):
 ...

Circuit breaker:
try:
 result = await agent.run(
 query, deps=deps
)
except UnexpectedModelBehavior as e:
 logger.error(
 f"Agent failed after retries: {e}"
)
Fallback или error response
return fallback_response(query)
```

# Graceful degradation в validation loops

Три уровня — от успеха до hard fail

## Успех

Ответ прошёл все уровни валидации — возвращаем типизированный результат

## Partial retry

1–2 попытки: ModelRetry с объяснением → модель исправляет → успех

## Max retries

3 попытки исчерпаны → UnexpectedModelBehavior → логируем + fallback

**Fallback = явный ответ с `partial_result: true`, не тихое молчание**

## Добавить validation loop в агента классификации

Задача: расширить агента из Практики #1 + #2 — добавить @output\_validator

Проверка 1: если priority = "critical" — requires\_human обязательно true

- **if output.priority == "critical" and not output.requires\_human: raise ModelRetry("critical tickets must require human review")**

Проверка 2: estimated\_minutes должно быть кратно 15

- **if output.estimated\_minutes % 15 != 0: raise ModelRetry(f"estimated\_minutes must be multiple of 15, got {output.estimated\_minutes}")**

Протестировать с тикетом уровня critical → убедиться что requires\_human = true

Проверить: retries срабатывают и ModelRetry сообщение информативно

# Live demo

colab

[demo](#)

[input](#)

# Live demo: результат

**5/5 инвариантов ОК** — оба правила валидатора соблюдены на всех тикетах:

- critical → requires\_human=True 
- estimated\_minutes % 15 == 0  (все значения: 15, 30, 15, 15, 30)

**2 retries сработали** — это ключевое доказательство что @output\_validator работает:

- Тикет 1 (взлом): модель с первого раза дала requires\_human=False или некратные минуты → валидатор поймал → ModelRetry → модель исправилась
- Тикет 2 (API 500): аналогично

## Итог Блока 4: validation loops как защитный слой агента

### **ModelRetry авто-ремонт**

Бросаем при логических ошибках

LLM видит проблему и исправляет

retries=3 — лимит

Сообщение = промпт  
для следующей итерации

### **output\_validator последний рубеж**

После Pydantic type check

Бизнес-логика, внешние API

Доступ к ctx.deps

UnexpectedModelBehavior  
после исчерпания лимита

### **3 уровня валидации**

1. Schema (автоматически)
2. @field\_validator (класс)
3. output\_validator (агент)

Каждый независим  
Errors с точным путём  
Graceful degradation

**4.7% → 0.0% логических ошибок при добавлении validation loops — реальный кейс юр.документов**

# БЛОК 4 – PRODUCTION

- Чеклист production-ready агента
- Топ-5 ошибок, метрики, ресурсы для изучения

# Production-ready агент: чеклист из 4 уровней

## Схема (контракт)

BaseModel с Field  
для всех выводов

Discriminated Union  
для success/error

Literal для  
перечислений

model\_json\_schema()  
для документации

## Deps (контекст)

@dataclass для  
зависимостей

Никаких секретов  
в промпте

Fake deps  
для тестов

RunContext  
типизирован

## Validation (защита)

@field\_validator  
для нормализации

@output\_validator  
для бизнес-логики

ModelRetry  
с контекстом

retries=3 + fallback

## Observability (мониторинг)

result.usage  
токены/стоимость

ctx.retry номер  
попытки

Logfire / Pydantic  
Logfire интеграция

Алёрт при retry>1

Все четыре уровня вместе — это архитектура агента, который не ломает prod

# Топ-5 ошибок при работе с PydanticAI

## Антипаттерны



### **BaseModel без Field(description)**

Модель не знает что ожидается в поле — quality падает, нужен отдельный промпт



### **Секреты и API-ключи в системном промпте**

Попадают в логи провайдера. Правильно: только в deps, никогда в строки промпта



### **ModelRetry с пустым или коротким сообщением**

«Invalid data» — модель не знает что исправить. Пиши как сообщение об ошибке



### **Нет retries-лимита и fallback**

При UnexpectedModelBehavior агент падает без ответа. Нужен circuit breaker



### **Глобальные переменные вместо deps**

Не работает при concurrent, нельзя протестировать. Правильно: RunContext[MyDeps]

# Метрики для мониторинга PydanticAI агентов

## Качество вывода

validation\_errors\_per\_run

retry\_count\_per\_run

Аномалия: > 1

fallback\_rate

Аномалия: > 5%

unexpected\_behavior\_rate

## Стоимость

tokens\_per\_run

(input + output)

cost\_per\_completion

retry\_overhead\_tokens

(токены потраченные  
на исправления)

retry\_cost\_%

## Latency

total\_run\_time\_ms

time\_per\_retry\_ms

tool\_call\_latency

p95 / p99 latency

Аномалия:

p99 > 30s при

max\_retries=3

# Ресурсы



## Документация и статьи

[ai.pydantic.dev](https://ai.pydantic.dev) — официальная документация PydanticAI

[pydantic.dev](https://pydantic.dev) «How to Use Pydantic for LLMs»

[realpython.com](https://realpython.com) «Pydantic AI: Build Type-Safe LLM Agents»

[abdullin.com](https://abdullin.com) — SGR Patterns и Examples

[tianpan.co](https://tianpan.co) «Structured Generation in Production»



## Инструменты и примеры

[github.com/pydantic/pydantic-ai](https://github.com/pydantic/pydantic-ai) — исходники + примеры

[logfire.pydantic.dev](https://logfire.pydantic.dev) — мониторинг агентов

[dev.to/pockit\\_tools](https://dev.to/pockit_tools) «LLM Structured Output in 2026»

[habr.com](https://habr.com) «Schema-Guided Reasoning» (русский)

# Спасибо за внимание

---

Вопросы? — 15 минут Q&A